Python Programming (CST-005)

Complete unit-wise notes - easy English, matched to the UTU syllabus.

Study Materials by Anas - study-materials-by-anas.vercel.app

Unit 1 - Introduction, Syntax & Data Types

What this unit covers: what Python is, how to run it, variables, data types, and basic input/output.

Key ideas: Python is **interpreted** (runs line by line), **dynamically typed** (no type declarations - the value decides the type), and uses **indentation** instead of braces. Core types: **int, float, str, bool**, plus the containers list, tuple, set, dict. Variables are names bound to objects; type() tells you the type, type casting (int(), str(), float()) converts. Input comes from input() as a **string** - cast it before arithmetic; output via print() with sep= and end= options. Comments use #, and Python is case-sensitive.

Exam pointers: short-answer favourites: "features of Python", "dynamic typing vs static typing", and predicting the output of type-casting snippets (especially input() without int()). Write one tiny program: read two numbers and print their sum, average and the larger one.

Unit 2 - Operators & Control Flow

What this unit covers: operators and control flow - decisions and loops.

Key ideas: Operators: arithmetic (+ - * / // % **), comparison (== != < > <= >=), logical (and or not), assignment (+= -= *= ...), membership (in), identity (is). Watch the difference: **/** always gives float, **//** floors, **** is power; **==** compares values, **is** compares identity. Control flow: **if / elif / else**; loops: **for** (over a sequence or range()) and **while** (condition-based). Loop control: **break** exits, **continue** skips to the next pass, and the loop-**else** runs only when the loop was not broken. range(start, stop, step) excludes the stop value.

Exam pointers: trace-the-output snippets with break/continue/else and nested loops are the usual questions; the full operator table in the notes below covers the table-format question. Practise writing: factorial, prime check, Fibonacci up to n terms, and a simple pattern print.

Extra note - Python Operators - the full table, made simple:

Type	Operators	Small example
Arithmetic	+ - * / // % **	`7 // 2 = 3`, `2 ** 3 = 8`
Comparison	== != > < >= <=	`5 > 3 = True`
Assignment	= += -= *= /=	`x += 2` means `x = x + 2`
Logical	and or not	`True and False = False`
Bitwise	& | ^ ~ << >>	`5 & 3 = 1` (bits ANDed)
Membership	in, not in	`'a' in 'anas' = True`
Identity	is, is not	`x is None`

Precedence (what runs first): brackets `()` first → power `**` → multiply/divide `* / // %` → plus/minus `+ -` → comparison → not → and → or last.

> *Just like BODMAS - brackets first, then power, then multiply-divide, then add-subtract.*

Exam tip: `//` is floor division (drops the decimal), `**` is power, and `is` checks identity while `==` checks value - these three confusions come up in every paper.

Unit 3 - Data Structures in Python

What this unit covers: Python's four built-in data structures - list, tuple, set, dictionary - and strings.

Key ideas: list: ordered, mutable, allows duplicates - append(), insert(), remove(), pop(), sort(), slicing [a:b:c]. **tuple**: ordered but **immutable** - faster, hashable, used for fixed records. **set**: unordered, unique items - union (|), intersection (&), difference (-). **dict**: key → value pairs; keys must be immutable and unique - get(), keys(), values(), items(), update(). **Strings** are immutable sequences: slicing, upper()/lower(), split(), join(), find(), strip(). A **list comprehension** like [x*x for x in range(10) if x%2==0] builds a list in one line - examiners love it.

Exam pointers: "list vs tuple", "list vs set vs dict" comparisons and method-based programs (count word frequencies with a dict, remove duplicates with a set) are the repeated questions. Slicing output questions (negative indices) catch people - practise a few by hand.

Unit 4 - Functions, Modules & Packages

What this unit covers: writing your own functions, organising code into modules and packages.

Key ideas: Define with **def name(params):** and send results back with **return**. Arguments: positional, keyword (name=value), **default** values, and variable-length ***args** (tuple) and ****kwargs** (dict). Python arguments are passed by object reference - mutable defaults bite, so remember the "don't use [] as a default" rule. **Scope:** local vs global (global keyword). **lambda** makes a one-line anonymous function; map/filter/reduce apply functions over sequences. Recursion: a function calling itself with a base case - factorial and Fibonacci are the standard examples. A **module** is a .py file you import; a **package** is a folder of modules. import forms: import math, from math import sqrt, import numpy as np.

Exam pointers: write-a-function questions (factorial by recursion, prime check, palindrome) and *args/**kwargs definitions are standard. Theory: actual vs formal parameters, local vs global with an example, and module vs package in one line each.

Unit 5 - File Handling

What this unit covers: file handling - reading from and writing to files, plus the usual exceptions around them.

Key ideas: Open a file with **open(name, mode)**: 'r' read (default, error if missing), 'w' write (creates/truncates), 'a' append, 'r+' read+write; add 'b' for binary. The **with open(...) as f:** block closes the file automatically - always prefer it. Read with read(), readline(), readlines() (list of lines) or just loop over f. Write with write()/writelines(). seek(0) moves the file pointer back to the start; tell() says where it is. Wrap file code in try/except to catch **FileNotFoundError** and IOError. CSV and JSON files appear often: the csv and json modules (json.load/json.dump) handle them.

Exam pointers: the standard program: read a text file and count lines/words/characters - practise it with the with-block. Short theory: file modes table, "why prefer with", and read vs readline vs readlines differences.

Made by Anas. Free forever. Watch the linked videos on the site after reading each unit.